

ACDSL: The Agentic Context Domain-Specific Language

Specification

2026-08-23 · version fdb221c

Contents

1. Abstract
 2. Introduction and motivation
 3. Design principles and anti-goals
 4. System model
 5. Language definition
 6. Operational semantics
 7. Where context is defined, and why
 8. Empirical evaluation
 9. Observability
 10. Problems and limitations
 11. Related work
 12. Future work
- Appendix A — Command-line reference
- Appendix B — Reading the live inventories
- Appendix C — Schemas
- Appendix D — Glossary

1. Abstract

ACDSL is a rule-gating and context-projection system for coding agents, built around a deliberately small declarative domain-specific language. Each rule is one declaration line in that language, binding a path anchor (a regular expression over repository files) to a registered verifier program and a human rationale. The language is declarative and non-Turing-complete by constitution — it carries no expressions, composition, or control flow; all executable semantics live in the registered verifier programs and the engine.

"ACDSL" names the language and, in everyday use, the whole system; this specification covers both and keeps the two layers distinct (§1.1). The system's central mechanism is *one rule source, two renderings*: every rule is a single declaration line rendered both as agent-visible context (a comment block projected into the files the rule governs) and as a deterministic, executable gate (a registered verifier program run against those same files). Because both renderings derive from the same line, they cannot drift from each other — the failure mode that kills conventionally maintained style guides and ontologies.

The system was built inside the `smine` repository, validated by six controlled A/B experiment rounds, and industrialized into a standing ablation loop in which every rule's projected prose is an experimental arm measured against gate verdicts. It currently governs doctrine over Go, shell, JSON, Markdown-plan, and skill-definition files; the live inventories are the files themselves (`acdsl/rules.acdsl`, `acdsl/registry.json`). ACDSL deliberately cannot express flow-sensitive or semantic properties: an expressibility audit fixed the ceiling at roughly half of the deepest-damage doctrine, accepted in exchange for guaranteed termination, second-scale gate latency, and a grammar an agent cannot subvert by argument.

1.1 The language and the system

- **The language** is the `//acdsl:` declaration grammar (§5): a concrete syntax with its own parser and escape rules, and domain semantics — anchor resolution over the git file universe, lifetime, reach, delivery — executed by the engine; even its ID grammar is gate-enforced (§5.2). Its family is the small declarative languages: CUE, Dhall, regular expressions, code-generation marker grammars.
- **The system around it** is everything that gives those declarations force: the gate engine (`check`, `fixtures`), the context projector (`project` plus the read hooks), the delivery experiment (`verdicts`, the `projected=` flag), and distribution (`dist`, `reach`).
- **Neither layer is** a programming language, a policy or governance engine, a reasoner, or a linter aggregator — external linters integrate through a documented seam (`docs/acdsl-go-tools.md`), and §11 names the rejected cluster.

2. Introduction and motivation

A coding agent is steered through two channels of fundamentally different strength.

Input-side context — style guides, instructions, examples in the prompt — the model *may* attend to; adherence is probabilistic and unobservable.

Output-side verification — deterministic programs inspecting what the agent actually produced — binds unconditionally: a violation is a red exit code regardless of what the model believed or forgot.

The founding claim is that *agentic reliability comes from the output side*. It replaced an earlier idea, examined and rejected during the system's conception: that context retrieval could be solved by ontology engineering. An ontology is a second source of truth, maintained beside the reality it describes. The two inevitably drift apart, and every drift turns into confidently wrong retrieval. ACDSL keeps no second source: a rule binds directly to the files it governs and the program that checks it, and both renderings derive from the same line.

ACDSL solves five problems:

- **Rule—enforcement drift.** Prose rule and enforcement are one artifact: a single declaration line renders as both the projected prose and the executed check.
- **Unbenchmarkable context.** A rule's prose can be withheld (`projected="false"`) while its gate keeps running; the per-arm violation rates *are* the experiment (§8).
- **Unbounded prose packs.** ACDSL installs a selection pressure: checkable rules migrate into gates, and the prose pack shrinks toward what genuinely requires judgment.
- **Cross-repository distribution.** A rule declares its deployment reach; `dist` ships the reach-covered slice — rules, registry subset, prebuilt binaries — to target repositories (§6.4), so doctrine is maintained once and enforced everywhere it applies.
- **Fragmented context layers.** Prose style rules, mechanical doctrine, and per-task contracts historically lived in separate systems. ACDSL unifies them: the checkable slice of prose migrates into gates (§7.3), doctrine rules and task contracts are the same object differing only in lifetime (§5.2), and every layer shares one identity grammar.

ACDSL is not a policy engine, governance framework, or reasoner (§11 names the rejected cluster). There is no inference over rules and no world modeling. A rule is a path pattern plus a program; the system's entire intelligence lives in the registered verifier programs, which are ordinary reviewed code.

3. Design principles and anti-goals

The principles that shaped the implementation, fixed in the founding idea document:

- **Narrow output spaces.** Constrain what the agent may produce rather than enriching what it reads.
- **Deterministic verifiers.** Model-as-judge is excluded from the gate path.
- **Verifier latency as a first-class axis.** Budgets are seconds — gates run inside agent retry loops whose economics collapse otherwise.
- **Quarantine, do not eliminate, the unverifiable.** Inexpressible doctrine stays prose, marked as judgment material, never force-fitted into a weak check.
- **Deterministic context assembly.** What the agent reads derives mechanically from the rule source.
- **Pack-level A/B against gate metrics.** No access to model internals required.
- **A usefulness gate on the system itself.** The prototype carried kill conditions.

Constitutional anti-goals, settled at inception: no inference over rules (rules do not interact), no contradiction/entailment machinery (two rules that disagree are an authoring problem), no policy or governance engine (ACDSL governs file contents, not agent permissions or actions), no world modeling (the only world represented is the git file universe, §6.1).

The DSL itself must be **declarative only**, **not Turing-complete**, **guaranteed to terminate** (every verifier runs under a hard timeout, §5.4), and **tiny** — uncontrolled grammar growth was identified as the design's principal failure mode.

4. System model

4.1 The rule

A rule is one declaration line binding four mandatory elements: **identity** + **anchor** + **verifier** + **context**.

- The **identity** is a stable ID under the grammar `KIND-SCOPE[-TOPIC]-NNN` (§5.2); verdict statistics, coverage metrics, and eviction decisions key on it.
- The **anchor** is a regular expression (Go RE2 syntax) over repository-relative file paths. It alone decides which files the rule governs — any file type, not only source code; where a rule is *declared* carries no meaning.
- The **verifier** is the name of a registered verifier program (§5.3–5.4; historically called *evalscript*). Only registered programs run; the registry is the trust boundary — *rules are data, verifiers are code*.
- The **context** is the rule's human-readable statement, carried in the `why=` field — the text that tells the agent (and the human) what the rule is. It is rendered into the projection and cited on every gate failure, so a red verdict always explains itself.

Optional elements modulate lifetime (§5.2), deployment reach (§6.4), prompt-side delivery (projected, §8.2), and verifier-specific parameters.

4.2 Assumptions

Five deliberate simplifications:

1. **The git file universe is the world model** — exactly `git ls-files --cached --others --exclude-standard` minus deleted-but-indexed paths (`internal/acdsl/run.go`). Untracked, unignored files are included: the gate must see what the agent just generated, before any commit.
2. **RE2 over paths is the only query language**. Content inspection is the verifier's job.
3. **The registry is the trust boundary**. Declarations (which agents may write, e.g. task contracts) cannot introduce executable behavior.
4. **Verifier determinism is a registration-time promise** made by the reviewing human, not framework-enforced (§10.6).
5. **One module per run**. Cross-repository consistency is handled by distribution (§6.4), not a federated check.

5. Language definition

5.1 Marker grammar

A rule declaration is one line in a `.acdsl` file — the language's own file type; doctrine is centralized in `acdsl/rules.acdsl`, task contracts live in their plan directories. Declarations are also recognized inline in every file class with a registered comment syntax (§6.2), each in its own marker form; inline declarations are reserved for rules that genuinely concern only their own file. The declaration body is identical everywhere:

```
MARKER ID SP VERIFIER { SP FIELD } [TERMINATOR]
FIELD      = KEY "=" QUOTED
QUOTED     = ''' { any character; \" and \\ are the only escapes } '''
```

The marker is the file class's own comment prefix, without inner spacing, plus `acdsl:` (internal/acdsl/rule.go, `declarationMarker`); block-comment classes close the line with their terminator:

File class	Marker	Terminator
<code>.acdsl, .go</code>	<code>//acdsl:</code>	—
<code>.sh, .py, .toml, .yaml, .yml, Makefile</code>	<code>#acdsl:</code>	—
<code>.sql</code>	<code>--acdsl:</code>	—
<code>.md, .html</code>	<code><!--acdsl:</code>	<code>--></code>

A fenced `//acdsl:` example inside a Markdown document is quoted text, never a declaration — Markdown declares only through its block-comment form. Unquoting deliberately resolves only `\` and `\\` — not the full Go escape set — so regular-expression escapes such as `\.go$` survive verbatim inside anchors.

Field	Requirement	Meaning
(<i>head, 1st token</i>) ID	required, unique per module	rule identity (§5.2)
(<i>head, 2nd token</i>) VERIFIER	required, must exist in the registry	the verifier to run
anchor	required	RE2 over repo-relative paths; selects governed files
anchor-not	optional	RE2 exclusion applied after <code>anchor</code>
why	required	rationale; projected and cited on failure
lifetime	optional, default <code>doctrine</code>	<code>doctrine</code> <code>task</code> (§5.2)
reach	optional, default <code>smine</code>	<code>global</code> <code>none</code> comma-separated repo-name list (§6.4)
projected	optional, default <code>true</code>	<code>false</code> = gate-only delivery (§8.2)
(<i>any other key</i>)	optional	verifier-specific parameter, passed as <code>key=value</code>

A malformed field, missing anchor/why, invalid field value, or duplicate ID is reported as a *violation* (exit 1), not a tool error — authoring mistakes surface through the same channel as rule breaches.

Centralizing doctrine in `acdsl/rules.acdsl` is a convention, not a language requirement — rules may live in any `*.acdsl` file.

5.2 Identity and lifetime

Identity grammar: `KIND-SCOPE[-TOPIC]-NNN`, e.g. `ACDSL-GOLANG-ERR-001`. Scope and topic segments must be registered in the shared taxonomy — the `aspects` array of the generated `context/context.json`, maintained through the aspect editor and regenerated only by `cmd/rules`; the grammar is itself gate-enforced (rule `ACDSL-SMINE-003`), so naming conformance is a mechanical check. Task-lifetime entries are exempt. Identity is load-bearing: verdict statistics key on it, so an ID migration deliberately restarts per-rule statistics.

Lifetime partitions rules into two populations with identical mechanics:

- `doctrine` (default) — standing house rules, gated on every `check` run.
- `task` — per-task contracts pinning a plan's promised artifacts (a symbol will exist with a given signature, a file's coverage will reach a threshold). They live in their own `*.acdsl` files inside plan

directories, are legitimately red mid-implementation, are gated explicitly via `check -lifetime` task, and are retired by renaming the file away from the `.acdsl` extension. House rules and task contracts are the same object, differing only in lifetime — a founding decision.

5.3 The registry

The verifier registry, `acdsl/registry.json`, maps a *name* to an invocation (`internal/acdsl/registry.go`). The name is the contract: a rule binds to the name, and any executable that honors the exit-code contract (§5.4) may stand behind it — the registry's `argv` is one implementation, not part of the language. A target repository overlays any entry by name via `acdsl/registry.local.json`, substituting its own implementation without touching the rules. Validation requires a non-empty `argv` and `timeout_s` in **1..60**.

```
"gofmt": {
  "argv": ["go", "run", "./cmd/acdsl/verifiers/gofmtwrap"],
  "timeout_s": 30,
  "description": "fails when an anchored file is not gofmt-formatted (...)"
}
```

A local overlay replacing the implementation behind the same name — any executable qualifies:

```
{
  "gofmt": {
    "argv": ["/tools/check-fmt.sh"],
    "timeout_s": 30,
    "description": "repo-local formatter check honoring the same contract"
  }
}
```

In the baseline instantiation every entry happens to invoke a small Go program via `go run` against a directory under `cmd/acdsl/verifiers/`; nothing in the engine assumes this — the engine executes `argv`, appends the files-list path and parameters, and reads exit codes. The per-entry `description` fields are the registry's own documentation (Appendix B).

5.4 The verifier contract

A verifier is any executable honoring one contract:

- **exit 0** — pass.
- **exit 1** — violations, one per stdout line as `file:line: message`. An exit-1 with no contract-shaped output still surfaces as a diagnostic — failures never vanish silently.
- **exit ≥ 2** — tool error, aborts the whole check run.

The engine invokes it as:

```
<argv...> <files-list-path> [key=value ...]
```

writing the anchored file set to a temporary list file and appending the rule's extra parameters in sorted-key order (`internal/acdsl/run.go`). Every invocation runs under `context.WithTimeout` at the entry's

`timeout_s`. Baseline verifiers needing external binaries shell out only through `internal/shell.Run` and only to allowlisted executables.

6. Operational semantics

6.1 The check pipeline

`acdsl check` executes, per rule surviving the lifetime and reach filters:

1. **Universe construction** — the git file universe (§4.2), computed once per run.
2. **Anchor resolution** — compile `anchor` (and `anchor-not`), filter the sorted universe (`internal/acdsl/anchor.go`). **An anchor resolving to zero files is an authoring error** — the typo guard: a doctrine rule silently governing nothing would be a vacuous green, and the contract refuses it. The single exception: for `lifetime="task"` rules, an empty anchor becomes a *diagnostic* ("planned artifact not yet present") — for a task contract, the missing file is the violation.
3. **Verifier execution** — under the entry's timeout, with the contract of §5.4.
4. **Diagnostics** — one line per violation: `message: [RULE-ID] verifier - why`; or JSON records with `-json`. The `why` travels with every failure.
5. **Verdict logging** — best-effort append to the verdict log (§9.1); a logging failure never changes the exit code.

Exit codes: `0` clean, `1` violations, `2` tool error. `check` additionally refuses staged projection blocks (§6.2) on every run. `-lifetime` defaults to `doctrine`; a task's contract is legitimately red mid-implementation and gated explicitly (`-lifetime task`, or `all`). `-rule <id>` gates a single rule of any lifetime — the dry-run used when validating a freshly generated verifier.

6.2 Projection

Projection renders the governing rules *into* the governed file, as a comment block directly above the content:

```
// [ACDSL-PROJECTION] 2 rule(s) govern this file - working-copy view, stripped before commit
// - [ACDSL-GOLANG-FMT-001] Every Go file is gofmt-formatted - ...
// - [ACDSL-GOLANG-PTR-001] Initialize pointers to composites with new(expr) - ...
```

What the agent reads is the projected file — view equals disk, so there is no separate context channel to fall out of sync. Semantics (`internal/acdsl/project.go`):

- **Syntax-table-driven.** The block is written in the file class's own comment syntax (`projectionSyntaxes`, the same table as §5.1's markers); insertion respects shebangs and Markdown frontmatter. Comment-incapable classes (JSON, `.acdsl`, binaries) are never touched on disk — their rules are *gate-only on disk*, delivered at read time by the `PostToolUse(Read)` hook `acdsl-context.sh` (`project -context`). Rules whose projection would break their own consumers carry `projected="false"`.
- **Working copy only.** `ValidateStagedClean` refuses staged projection content whose comment shape matches the file's own syntax (a doc quoting another syntax's form is a reference, not a leak); `project -strip` removes every block before committing (wired into `make audit`). The marker literal never appears in committed code.
- **Idempotent.** `project -file` inserts, refreshes, or removes the block to match the current rule set; a rule flipped to gate-only self-heals on the next read.

- **Plan-time resolution.** `project -plan <path>` prints the rules that *would* govern a path that need not exist yet — pure regex, no universe membership (`internal/acdsl/scope.go`); design plans record their planned files' rules through this verb.

6.3 Fixtures

Every rule should carry committed pass/fail example sets under `acdsl/testdata/<rule-id>/`. `acdsl fixtures` proves each rule's verifier against them: the fail set must produce violations, the pass set must not. Fixtures are the gate's own test suite — a prerequisite for eviction decisions (§8.3).

6.4 Reach and distribution

`reach=` declares a rule's deployment reach (`internal/reach` grammar): `global` (would bind in any repository), `none` (disabled: `check` skips it, it never projects, it ships nowhere — fixtures stay runnable so re-enabling is cheap), or a comma-separated list of arbitrary target repository names — e.g. the live `reach="go"` binds a rule to the repository named `go` alone. The home repository is named `smine` (the default) — an ordinary name that never matches a sync target, so there is no self-keyword.

`acdsl dist -target <name> -dest <dir>` ships a target's gate slice (`internal/acdsl/dist.go`): the reach-covered doctrine rules as verbatim marker lines under a synced-from-smine header, the registry subset those rules name with `argv` rewritten to `bin/verifiers/<name>`, and prebuilt binaries. The no-match contract stands everywhere: `dist` ships a rule only when the target has files its anchor matches; skipped rules are reported and picked up by a later re-sync. `Dist` refuses to overwrite a repository-owned (unheadered) rules file. Targets own their side: additional rules in any `*.acdsl` file, and verifier overrides in `registry.local.json`.

6.5 Enforcement integration points

Point	Mechanism	Effect
Agent read time	PreToolUse hook on Read (<code>cmd/hooks/acdsl-project.sh</code>)	projects the file on disk before the agent reads it; no-ops unless <code>acdsl/registry.json</code> exists; prefers <code>bin/acdsl</code> over <code>go run</code> ; never blocks a read ; toggleable via <code>ACDSL_PROJECT_ENABLED</code>
Agent read time, non-projectable files	PostToolUse hook on Read (<code>cmd/hooks/acdsl-context.sh</code>)	injects the governing rules for comment-incapable files (JSON, <code>.acdsl</code>) as <code>additionalContext</code> via <code>project -context</code> ; toggleable via <code>ACDSL_CONTEXT_ENABLED</code>
Local audit	<code>make audit</code>	<code>go mod verify + go vet</code> , then <code>project -strip → check → fixtures</code> , then tests; the gates replace the raw Makefile lines they subsume
Smoke test	<code>cmd/tests/test_acdsl.sh</code>	asserts expected rule/fixture counts; exports <code>ACDSL_VERDICTS_ENABLED=0</code> so deliberate reds do not pollute statistics
Continuous integration	<code>.github/workflows/ci.yml</code>	runs <code>make test</code> (includes the smoke) but not <code>make audit</code> — <code>check / fixtures</code> are not directly a CI gate today; CI builds the binaries for the distribution payload
Commit time	<code>none</code>	no pre-commit hook runs <code>check</code> ; the staged-projection guard and the <code>make audit</code> discipline are the commit-hygiene mechanisms

Agent write access to the rule surface is permission-gated: `Edit/Write` on `**/*.acdsl` and `acdsl/**` require explicit user approval (the agent-lock, `settings/claude_code/settings.json`), so an agent cannot silently weaken the rules that gate it.

7. Where context is defined, and why

ACDSL exists inside a larger context architecture; *where* guidance lives is a design decision with measurable consequences. The surfaces differ on binding strength, scope, and token cost.

- **7.1 ACDSL projection blocks — per-file, mechanical, gate-backed.** The Read-hook projection (§6.2): exactly the anchored files, at the moment the agent reads them, at ~0.1k tokens per governed file — two orders of magnitude cheaper than pack delivery (~20k tokens, §8.1). Binding is the strongest available: the same line is enforced by a deterministic gate. The destination surface for every rule that clears the checkability bar.
- **7.2 The global context pack — per-session judgment and facts.** A session-start (and subagent-start) hook (`cmd/hooks/global-context.sh`) injects the generated context index (`context/context.json`). Judgment doctrine and repository facts have no anchor; session start is the only universal delivery point. Binding: none. *Known weakness*: the pack is injected whole regardless of what the session does; §7.4 is the designed correction.
- **7.3 Language style rules — judgment prose with a migration pressure.** Per-language guides (`context/rules/*.md`), delivered by the read-gate hook (`cmd/hooks/read-gate.sh`): it maps a touched file to its governing guide via the file-glob index in `context/context.json` and *denies* the Read/Edit/Write until the guide has been read in full — the first touch of a language forces its guide into context. The read is compelled; adherence is not, and the checkable slice migrates into gates. The measured constraint (§8.1): prose-only rule statements die; judgment rules must carry worked examples.
- **7.4 Skill `acdsl-context: declarations` — per-skill, demand-scoped.** A skill's frontmatter declares the context entries it needs (the key is `acdsl-namespaced` — bare `context:` is Claude Code's native field); the skill-context hook injects exactly those at invocation. Resolvability is itself gated (ACDSL-SKILL-004/005). The skill becomes a projection target — ACDSL's model (anchored, declared, gate-validated context) applied to prose delivery.
- **7.5 Task contracts — per-plan, existence-checked.** `lifetime="task"` rules (§5.2), typically via `symbol-exists`, `symbol-coverage`, `golden-diff`. Full gate strength; red until the promised artifact exists, retired with the plan. The need is measured (§8.1): blind implementers renamed planned symbols in every run, contract-carrying runs in none.
- **7.6 The declaration site — an accidental channel.** Probe agents navigated to a rule's home package and read the inline marker itself, contaminating the first experiment's control arm — one motivation for centralizing doctrine in `acdsl/rules.acdsl`: making the channel explicit and controllable.

The governing principle across all surfaces: **context shrinks toward judgment rules and facts; everything mechanical lives as gates.** New guidance is placed by asking, in order: checkable (→ 7.1)? a plan promise (→ 7.5)? activity-specific judgment (→ 7.4, with worked examples per 7.3)? Only then the global pack (7.2) — the weakest binding at the highest standing cost.

8. Empirical evaluation

8.1 Findings from the controlled rounds

Six experiment rounds (probe agents implementing seeded tasks under varying context arms; designs and data in `evals/acdsl-ab-2026-08/ab*_results.md`):

- **The differential concentrates where doctrine diverges from model priors.** On an anti-prior rule with no in-repo precedent: 0/6 first-pass conformance without any rule channel, 6/6 with one. Where repo structure already teaches the rule, every arm reaches ceiling.
- **Burial in volume is free; delivery cost is not.** The same rule buried in a ~20k-token pack still achieved 20/20 adherence — but the pack costs ~20k tokens per run against projection's ~0.1k per file. Burial *in time* remains untested (§10.4).
- **Worked examples are load-bearing.** Rule form, not task affinity, predicts survival: the prose-only bullet died 0/10, worked-example rules survived 13/15.
- **Prose functions as a behavioral instruction.** Removing a single sentence collapsed a tool-discipline rule from full to near-zero conformance; every red converted in exactly one mechanical retry (~28k tokens each) — the strongest evidence for an autofix ladder (§12).
- **Task contracts close the plan-drift hole.** Blind implementers renamed the planned symbol in 100% of runs; contract-carrying runs missed 0%. Adversarial probes also pasted marker lines into source — leak-copy as gate denial-of-service (§10.7).

The cross-round invariant: every gate-visible violation converted to green in one generated retry — the gate-plus-retry loop is economically sound at current rule counts.

8.2 The standing ablation loop

The delivery flag `projected="false"` removes a rule's prose from every prompt-side channel — projection blocks and plan-time resolution — while `check` and `fixtures` never consult the flag: *enforcement is the constant; delivery is the variable*. Prompt-side output stays pure — no hint that a rule is also enforced, since such a note would itself be a delivery channel and contaminate the arm. Verdict statistics are keyed on the pair *(rule, projected)* (`internal/acdsl/verdicts.go`); a rule flipped mid-window shows two rows — that pair is the A/B comparison. A flip is one field edit plus a commit; there is deliberately no delivery toggle in the user interface.

8.3 Eviction and re-projection policy

The policy encodes a deliberate asymmetry (the original policy demoted rules with low red rates — precisely when they caught rare failures — and was inverted after review, §10.8): *any red run is proof the rule earns its projection* — ten 1% failures are ten fully failed features, so a low red rate is never an eviction argument. A rule qualifies for `projected="false"` only when **both** hold: its verifier and fixtures are green, and the projected arm shows **≥ 300 logged runs with zero reds**. Re-projection criterion: **any red on the gate-only arm** flips the rule back. Cadence: a monthly `verdicts -since 720h` review. The user interface flags both candidate classes but never flips. The numbers are an initial calibration, not measured constants — the ≥ 300-run bar in particular is deliberately arbitrary and subject to adjustment as the loop accumulates data (§10.9).

9. Observability

The verdict log. Every `check` run that reaches verifier execution appends one JSONL record (`internal/acdsl/verdicts.go`; schema in Appendix C). Sink: `$ACDSL_VERDICTS_PATH`, else `~/.claude/acdsl/verdicts.jsonl` — home-anchored deliberately, because pool worktrees are destroyed with their sessions and the loop's data must outlive them. Zero-violation rows are deliberate (the rate denominator); the git branch is the session proxy, with `CLAUDE_SESSION_ID` when present. Logging is best-effort — the gate's exit code never depends on it; `ACDSL_VERDICTS_ENABLED=0` disables it (exported by the smoke suite, whose deliberate reds would poison the statistics). `acdsl verdicts` aggregates the log per (rule, projected): runs, reds, violations, last-red.

The user interface. The configuration server's Context tab (`internal/server/acdsl.go`) shows rule cards with delivery-state badges and per-arm red rates, flags flip candidates per §8.3 without ever flipping, and derives a coverage metric from rules whose `why` cites the context entry they enforce. `reach` is editable there; delivery deliberately is not.

The session-mining feedback loop. Mined context proposals carry an enforcement-band classification; sketched gates are generated within the verifier-generation bounds (`acdsl/evalgen.json`); mining joins against `acdsl verdicts`; and mined dispositions map onto ACDSL actions — new rules, projected flips, or eviction proposals (which require the ≥ 300 -clean-run bar).

10. Problems and limitations

Measured or designed-in boundaries, not speculation.

- **10.1 Overfitting.** An agent-drafted corpus gravitates toward what is easy to check about the *current* tree; the first corpus was rejected wholesale as checkable-but-irrelevant (`evals/acdsl-ab-2026-08/corpus.md`). The defense — value filter before checkability, human veto — is procedural, not mechanical.
- **10.2 Underfitting: the expressibility ceiling.** Half the damage-rule corpus is expressible; the inexpressible half — goroutine ownership, clock transitions, hidden I/O — needs analysis the anti-goals refuse (`evals/acdsl-ab-2026-08/audit.md`) and stays prose-governed judgment (§7.3). External linters raise the ceiling only marginally, at the cost of a mandatory disable list (`docs/acdsl-go-tools.md`).
- **10.3 Context bloat.** Full-pack delivery measured ~52k tokens per probe against projection's ~0.1k per file, and the session hook still injects the whole pack unconditionally (§7.2); no mechanism caps rule count.
- **10.4 Burial-in-time.** The rule delivered at turn 1 with the artifact generated at turn 40 is untested — the largest unmeasured threat to the input side. Projection partially sidesteps it (the block re-arrives on every read); plan- and pack-carried context does not.
- **10.5 Verifier latency.** `go run` compiles on every check; no caching, no batching — per-rule attribution runs N processes for N rules, and the 60-second ceiling collides with heavyweight engines.
- **10.6 Verifier reliability.** Determinism is promised, not enforced — a network-touching or time-dependent verifier would corrupt verdict statistics silently. A weak primitive can green-light files it did not really check; a rule without a failing fixture is unproven.
- **10.7 Adversarial and hygiene surface.** Models copy projection or marker lines into source, and a malformed copy aborts the entire check at parse time — the guards mitigate, the parse-abort stays a single point of failure. Probe/test runs pollute the verdict sink unless `ACDSL_VERDICTS_ENABLED=0` is exported.

- **10.8 Process risks.** The original eviction policy demoted rules precisely when they caught rare failures and had to be inverted (§8.3) — verdict metrics must be read against the cost asymmetry of agent failures, not raw rates. Nothing kills drift between the rule and reality; attention remains unobservable.
- **10.9 Non-stationary environment.** The rule set churns continuously, so the ablation arms never run against a fixed environment: the statistics are rough guidance, thresholds like the eviction bar provisional. What the loop reliably delivers is visibility — a heavily violated rule is unmissable.

11. Related work

The design defines itself by negation: the examined-and-rejected cluster — AgentSpec, Cedar/OPA/Rego, RAIL/Colang, SHACL, ontology engineering — shares rich rule interactions, inference engines, and a maintenance surface that grows with expressiveness. ACDSL's wager is the opposite corner: a grammar too small for scale explosion, semantics too shallow for a reasoner to disagree with, and all expressive power delegated to reviewed programs behind a named-registry trust boundary. The nearest conventional relatives are lint-configuration formats, from which ACDSL differs by the projection half (rules travel to the model as context) and the ablation half (delivery is an instrumented experimental variable).

12. Future work

From the v1 roadmap and the round findings:

- **The input/output contract reframe:** treat the agent as an unreliable black box with fully controlled input and output parameters — universal anchoring of every input surface, skills closing the loop with schema-validated reports.
- **The autofix ladder:** a `fix` argv per registry entry, `check` → `autofix` → `recheck`, removing mechanical failure classes from the model economy entirely.
- **Fine-grained delivery:** per-symbol rather than per-file projection — deferred, demand-gated.
- **Infrastructure debt and further experiments:** prebuilt verifier binaries on the check path, caching, batch-demultiplexing with preserved per-rule attribution; burial-in-time; further rounds on a parallelized, worktree-isolated harness.

Appendix A — Command-line reference

Source: `cmd/acdsl/main.go`. Exit codes: 0 clean, 1 violations, 2 tool error.

Command	Flags	Function
<code>acdsl check</code>	<code>-json</code> , <code>-lifetime doctrine task all</code> (default <code>doctrine</code>), <code>-rule <id></code> , <code>-registry <file></code> , <code>-root <dir></code>	resolve anchors, run verifiers, print violations, refuse staged projection blocks, log verdicts (best-effort)
<code>acdsl project -file <path></code>	<code>-root <dir></code>	sync one file's on-disk projection block (insert/refresh/remove)
<code>acdsl project -strip</code>	<code>-root <dir></code>	remove every projection block (pre-commit sweep)
<code>acdsl project -plan <path></code>	<code>-root <dir></code>	print rules that would govern a path that need not exist yet
<code>acdsl project -context <path></code>	<code>-root <dir></code>	print the delivered rules for a file with no comment syntax (the PostToolUse read hook's data source); empty for projectable files
<code>acdsl fixtures</code>	<code>-lifetime</code> , <code>-registry</code> , <code>-root</code>	prove each rule's verifier against <code>acdsl/testdata/<rule-id>/ pass/fail</code> sets
<code>acdsl verdicts</code>	<code>-path <file></code> (default <code>\$ACDSL_VERDICTS_PATH</code> or <code>~/.claude/acdsl/verdicts.jsonl</code>), <code>-since <duration></code>	aggregate the verdict log per (rule, projected): runs, reds, violations, last-red
<code>acdsl dist</code>	<code>-target <name></code> (required), <code>-dest <dir></code> (required), <code>-root <dir></code> , <code>-task</code> (include reach-covered task-lifetime rules)	ship a target's gate slice: reach-covered rules, rewritten registry subset, prebuilt binaries

Environment: `ACDSL_VERDICTS_PATH` (sink override), `ACDSL_VERDICTS_ENABLED=0` (disable logging), `ACDSL_PROJECT_ENABLED` (projection-hook toggle), `ACDSL_CONTEXT_ENABLED` (context-hook toggle).

Appendix B — Reading the live inventories

The spec carries no copies of the rule or verifier inventories; they change with the repository and are authoritative only at their sources:

- **Doctrine rules** — `acdsl/rules.acdsl`, one declaration line per rule; the file's header comment documents the local conventions. Rules with `reach="global"` form the distribution manifest.
- **Verifier registry** — `acdsl/registry.json`: `name → {argv, timeout_s, description}`. The `description` field is the per-entry documentation; the `argv` is the only name ↔ implementation mapping. `acdsl/registry.local.json` overlays entries by name in a target repository.
- **Fixtures** — `acdsl/testdata/<rule-id>/ pass/fail` sets; `acdsl fixtures` proves them. A rule without a failing fixture is unproven (§10.6).

A handful of registry entries (the task-contract and observation primitives, e.g. `symbol-exists`, `golden-diff`) deliberately have no doctrine consumer.

Appendix C — Schemas

Verdict record (`internal/acdsl/verdicts.go`), JSONL, one per check run reaching verifier execution:

```

VerdictRecord { ts: RFC3339, root: string, branch: string, session?: string,
                outcome: string, rules: [RuleVerdict], diagnostics?: [DiagnosticRef] }
RuleVerdict   { id: string, projected: bool, violations: int }
DiagnosticRef { id: string, message: string }

```

Registry entry (`internal/acdsl/registry.go`): { `argv: [string]` (non-empty), `timeout_s: int` (1..60), `description: string` }; `registry.local.json` overlays by name.

Appendix D — Glossary

Core terms (anchor, verifier, why, gate, projection, file universe) are defined at first use in §4–§6; the loop vocabulary:

Term	Meaning
Ablation loop	the standing experiment: per-rule prose delivery as an instrumented variable against gate verdicts
Band (F/A/D/J/G)	enforcement-cost classification: free tool / cheap AST / dependency-gated / judgment prose / already gated
Delivery flag	<code>projected=</code> — whether a rule's prose reaches prompt-side channels
Eviction	flipping a rule to gate-only after ≥300 clean projected runs
Leak-copy	a model copying projection or marker text into source files
Reach	a rule's deployment scope across repositories (<code>global</code> / <code>none</code> / name list)
Task contract	a per-plan rule (<code>lifetime="task"</code>) pinning a promised artifact
Verdict log	the JSONL record stream of check outcomes, keyed per (rule, projected)